Metinler (str)

İndeksleme • dilimleme • metin metotları • metin üzerinde yineleme • fonksiyon kavramına giriş

7. hafta • 2 teori + 2 uygulama

Bu haftanın hedefleri

Ders sonunda ne yapabiliyor olacağız?

1 • Erişim

Metindeki karakterlere indeks ile ulaşmak

2 • Parça alma

Dilimleme ile metnin belirli bölümünü seçmek

3 • İşleme

Temel str metotlarıyla metni dönüştürmek / sorgulamak

4 • Yineleme

for ile karakterleri tek tek işlemek

4 ders saatlik yol haritası

Anlatım ve uygulama birlikte ilerleyecek

1. saat

str mantığı
indeksleme
len()

2. saat

dilimleme
metin metotları
normalizasyon

3. saat

for + str
karakter analizi
bütünleşik örnekler

4. saat

fonksiyon fikri
ara sınav tekrarı
hata avı

Geçen haftadan bugüne

Döngü artık yalnızca sayılar üzerinde çalışmayacak

Geçen hafta

```
for sayi in range(...):
```

Bu hafta

```
for karakter in metin:
```

01

Metin (str) nedir?

Karakterlerden oluşan bir veri türünü adım adım inceleyelim.

String: karakter dizisi gibi düşünün

Bir metin tek parça görünür; Python karakterleri sırayla tutar

P	y	t	h...
0. karakter	1. karakter	2. karakter	devam eder

String değer oluşturma

Tek veya çift tırnak kullanılabilir

```
ad = "Mert"
sehir = 'İstanbul'
mesaj = "Python öğreniyorum"

print(type(ad))
```

Tırnak içindeki değer → str

len(): metnin uzunluğu

Boşluk ve noktalama işaretleri de karakterdir

len("Python")

6

len("AI 101")

6

len("")

0

İndeks: karakterin konumu

Python saymaya 0’dan başlar

0	1	2	3…
P	y	t	h…

Pozitif indeksleme

Köşeli parantez ile tek karakter seçilir

```
metin = "Python"

print(metin[0])
print(metin[1])
print(metin[5])
```

Tek karakter → str türünde bir sonuç

Negatif indeksler

Sondan geriye doğru saymak için

-1

son karakter

-2

sondan ikinci

-3

sondan üçüncü

Aynı karaktere iki yoldan erişim

Pozitif ve negatif indeksler birlikte

```
metin = "Python"

print(metin[5])   # n
print(metin[-1])  # n
print(metin[0])   # P
print(metin[-6])  # P
```

Aynı karakter • farklı indeks

Sınırın dışına çıkarsak?

IndexError

```
metin = "Python"

print(metin[6])
```

Hata mesajını okuyun; programın ne beklediğini söyler.

Tahmin et → çalıştır → karşılaştır

Sınıf içi düşünme / uygulama

metin = "YapayZeka"

metin[0] • metin[4] • metin[-1] • len(metin)

Önce kağıt üzerinde karakterleri indeksleyin.

Mini uygulama 1

Sınıf içi düşünme / uygulama

Kullanıcının adının ilk ve son karakterini yazdırın.

input() • [0] • [-1] • len()

02

Dilimleme (slicing)

Tek karakter değil, metnin bir bölümünü seçelim.

Dilimleme sözdizimi

metin[başlangıç:bitiş:adım]

başlangıç

dahil

bitiş

hariç

adım

kaçarlı ilerle?

Başlangıç:bitiş

Bitiş indeksi sonuca dahil değildir

```
metin = "Python"

print(metin[0:3])  # Pyt
print(metin[2:5])  # tho
print(metin[1:6])  # ython
```

Kural: başlangıç dahil • bitiş hariç

Sınırları boş bırakabiliriz

Python metnin başını veya sonunu varsayar

metin[:3]

baştan 3’e kadar

metin[2:]

2’den sona kadar

metin[:]

metnin tamamı

Adım değeri

Kaçar kaçar ilerleyeceğimizi belirler

```
metin = "0123456789"

print(metin[0:10:2])  # 02468
print(metin[1:10:2])  # 13579
```

Adım yazılmazsa varsayılan değer 1

Metni ters çevirme

Negatif adım: sağdan sola ilerleme

```
metin = "Python"

ters = metin[::-1]
print(ters)  # nohtyP
```

Kısa yol: metin[::-1]

İndeksleme ≠ dilimleme

Sınır dışı davranışları farklıdır

metin[99]

```
IndexError
```

metin[:99]

```
çalışır
```

Uygulama: tarih metnini parçalama

Henüz datetime yok; sadece string dilimleme

```
tarih = "2026-09-19"

yil = tarih[:4]
ay = tarih[5:7]
gun = tarih[8:]

print(yil, ay, gun)
```

Sabit format → sabit karakter konumları

Mini uygulama 2

Sınıf içi düşünme / uygulama

“MERT HIZMALI” metninden adın ilk 2, soyadın son 3 karakterini alın.

İndeksleri önce kağıt üzerinde belirleyin.

03

String metotları

Metni değiştiren değil, yeni sonuç üreten hazır araçlar.

Fonksiyon mu, metot mu?

Çağrı biçimine bakın

Fonksiyon

```
len(metin)
```

Metot

```
metin.upper()
```

upper() ve lower()

Büyük / küçük harf normalizasyonu

```
metin = "PyThOn"

print(metin.upper())  # PYTHON
print(metin.lower())  # python
print(metin)          # PyThOn
```

Metot sonucu yeni bir stringdir

strip(): kenar boşluklarını temizler

Baştaki ve sondaki boşluklar

```
kullanici = "   mert29   "

temiz = kullanici.strip()

print(kullanici)
print(temiz)
```

Temiz veri → daha güvenilir karşılaştırma

replace(): belirli parçayı değiştirir

Eski → yeni

```
metin = "python dersi"

yeni = metin.replace("python", "Python")
print(yeni)
print(metin)
```

replace() orijinali değil sonucu değiştirir

find(): aranan parçanın konumu

Bulursa indeks • bulamazsa -1

```
metin = "programlama"

print(metin.find("gram"))
print(metin.find("Python"))
```

find() → int

count(): kaç kez geçiyor?

Sonuç bir tam sayıdır

```
metin = "programlama"

print(metin.count("a"))
print(metin.count("m"))
print(metin.count("x"))
```

Frekans sayımı için hızlı araç

startswith() / endswith()

Sonuç: True veya False

startswith()

Belirli bir parçayla başlıyor mu?

endswith()

Belirli bir parçayla bitiyor mu?

Karakter içeriğini sorgulama

Basit girdi doğrulama araçları

isalpha()

yalnızca harflerden mi oluşuyor?

isdigit()

yalnızca rakam karakterleri mi?

isalnum()

harf veya rakam mı?

Stringler immutable’dır

Bir karakteri doğrudan değiştiremeyiz

```
metin = "Python"

# HATALI:
# metin[0] = "J"

metin = "J" + metin[1:]
print(metin)  # Jython
```

Değiştirmek yerine yeni değer üretiriz

Zincirleme: temizle + küçült

Bir metodun çıktısı diğerine gider

```
kullanici = "   MERT29   "

temiz = kullanici.strip().lower()
print(temiz)  # mert29
```

Sıra önemlidir: önce strip(), sonra lower()

Uygulama: kullanıcı adı normalizasyonu

Girdiyi standart hale getir

```
kullanici = input("Kullanıcı adı: ")

kullanici = kullanici.strip().lower()
kullanici = kullanici.replace(" ", "_")

print(f"Kaydedilen ad: {kullanici}")
```

Normalizasyon = aynı anlama gelen girdileri ortak biçime getirmek

Uygulama: basit kod doğrulama

Uzunluk + başlangıç + rakam kontrolü

```
kod = input("Kod: ").strip()

gecerli = len(kod) == 6 and kod.startswith("A") and kod[1:].isdigit()

print(f"Geçerli mi? {gecerli}")
```

Bir problem → birden fazla önceki konu

Mini uygulama 3

Sınıf içi düşünme / uygulama

Bir e-posta metnini temizleyin ve “@” içerip içermediğini kontrol edin.

strip() • lower() • in

04

Metin üzerinde arama ve yineleme

in / not in • for ile karakter karakter işlem

in / not in

Bir karakter veya alt metin var mı?

```
metin = "python programlama"

print("python" in metin)
print("java" in metin)
print("x" not in metin)
```

Sonuç: bool

for ile string üzerinde dolaşma

Her turda sıradaki karakter gelir

```
metin = "AI"

for karakter in metin:
    print(karakter)
```

for değişkeni → sıradaki karakter

Döngü izleme

metin = "KOD"

1. tur

karakter = "K"

2. tur

karakter = "O"

3. tur

karakter = "D"

Sesli harf sayacı

for + if + sayaç

```
metin = input("Metin: ").lower()
sesli_sayisi = 0

for karakter in metin:
    if karakter in "aeıioöuü":
        sesli_sayisi += 1

print(sesli_sayisi)
```

Algoritma: dolaş → kontrol et → gerekiyorsa say

Metindeki rakamları sayma

isdigit() her karakter üzerinde

```
metin = input("Metin: ")
rakam = 0

for karakter in metin:
    if karakter.isdigit():
        rakam += 1

print(f"Rakam sayısı: {rakam}")
```

Hazır count() yerine algoritmayı kendimiz kuruyoruz

Hedef karakteri sayma

count() mantığını kendimiz yazalım

```
metin = input("Metin: ").lower()
hedef = input("Aranan harf: ").lower()
sayac = 0

for karakter in metin:
    if karakter == hedef:
        sayac += 1

print(sayac)
```

Hazır araç ile algoritmik çözümü karşılaştırın

Yeni metin oluşturma: boşlukları atla

String immutable → yeni string biriktir

```
metin = input("Metin: ")
yeni = ""

for karakter in metin:
    if karakter != " ":
        yeni += karakter

print(yeni)
```

Biriktirici her zaman sayı olmak zorunda değildir

Döngüyle metni ters çevirme

Karakteri yeni metnin başına ekle

```
metin = input("Metin: ")
ters = ""

for karakter in metin:
    ters = karakter + ters

print(ters)
```

Her turda ara değeri izleyin

Palindrom kontrolü

Temizle → ters çevir → karşılaştır

```
metin = input("Metin: ").strip().lower()
ters = metin[::-1]

if metin == ters:
    print("Palindrom")
else:
    print("Palindrom değil")
```

Birden fazla konu → tek problem

Bütünleşik örnek: metin analiz raporu

Uzunluk • harf • rakam • boşluk

```
metin = input("Metin: ")
harf = 0
rakam = 0
bosluk = 0

for karakter in metin:
    if karakter.isalpha():
        harf += 1
    elif karakter.isdigit():
        rakam += 1
    elif karakter == " ":
        bosluk += 1

print(len(metin), harf, rakam, bosluk)
```

Tek geçişte birden fazla ölçüm

Mini uygulama 4

Sınıf içi düşünme / uygulama

Bir metindeki sesli, rakam ve boşluk sayılarını ayrı ayrı bulun.

for • if/elif • in • isdigit()

05

Fonksiyon kavramına giriş

Bugün yalnızca fikir ve temel çağrı mantığı; ayrıntılar ileride.

Zaten fonksiyon kullanıyoruz

İsim + parantez + gerektiğinde değer

print()

ekrana çıktı

input()

kullanıcıdan veri

len()

uzunluk sonucu

type()

veri türü

Fonksiyon ve metot: aynı şey değil

Ama ikisi de hazır davranış çağırır

len(metin)

`fonksiyon çağrısı`

metin.upper()

`string metodu`

Neden fonksiyon?

Aynı işi tekrar tekrar yazmamak için

Tekrar

aynı kodu kopyalama

İsimlendirme

işe anlamlı bir ad verme

Yeniden kullanım

gerektiğinde çağırma

İlk bakış: kendi fonksiyonumuz

Ayrıntılı fonksiyonlar 11. haftada

```
def cizgi_yaz():
    print("-" * 30)

print("RAPOR")
cizgi_yaz()
print("Tamamlandı")
```

Bugün: kavramı tanı • sonra ayrıntılandıracağız

06

Ara sınav öncesi bütünleşik tekrar

İlk 7 haftayı kod okuma, çıktı tahmini ve hata avıyla birleştirelim.

Tekrar 1 • Veri türü + operatör

Çıktıyı çalıştırmadan tahmin edin

```
a = "5"
b = 2

print(a * b)
print(int(a) + b)
```

Önce veri türünü belirle • sonra operatörü yorumla

Tekrar 2 • if / elif sınırı

Hangi dal çalışır?

```
puan = 70

if puan > 70:
    print("A")
elif puan >= 50:
    print("B")
else:
    print("C")
```

Sınırdaki eşitlik işaretini kaçırmayın

Tekrar 3 • while izleme

Sayaç ve koşul

```
sayi = 1

while sayi < 5:
    print(sayi)
    sayi += 2
```

while: başlangıç → koşul → gövde → güncelleme

Tekrar 4 • for + range

Bitiş dahil değil

```
toplam = 0

for sayi in range(1, 6, 2):
    toplam += sayi

print(toplam)
```

Önce range değerlerini yaz • sonra döngüyü izle

Tekrar 5 • String + for + if

Kaç tane “a” var?

```
metin = "Ankara".lower()
sayac = 0

for harf in metin:
    if harf == "a":
        sayac += 1

print(sayac)
```

Kodun ilk satırı döngünün verisini değiştirebilir

Hata avı

Kod çalışıyor mu? Mantık doğru mu?

```
metin = input("Metin: ")
sesli = 0

for harf in metin:
    if harf == "a" or "e":
        sesli += 1

print(sesli)
```

Çalışması, doğru olduğu anlamına gelmez

Hata avı 2 • Son karakter

Off-by-one

```
metin = "Python"
son = metin[len(metin)]
print(son)
```

Uzunluk ≠ son indeks

Bütünleşik sınav tipi soru

Kullanıcı kodu sınıflandırma

```
kod = input("Kod: ").strip().upper()

if len(kod) == 5 and kod[0] == "A" and kod[1:].isdigit():
    print("Geçerli A kodu")
else:
    print("Geçersiz")
```

Karmaşık kodu küçük koşullara ayırın

Kod okuma stratejisi

Sınavda sistematik ilerleyin

1

Değişkenleri ve türleri yaz

2

İfadeleri sırayla değerlendir

3

Koşul / döngü akışını izle

4

Çıktıyı en son yaz

Hata ayıklama kontrol listesi

Hata mesajı + veri + akış

Syntax

Kod yazım olarak geçerli mi?

Runtime

Çalışırken sınır / dönüşüm hatası var mı?

Logic

Çalışıyor ama sonuç yanlış mı?

Haftanın bireysel uygulaması

Sınıf içi düşünme / uygulama

“Metin Analiz Aracı” geliştirin.

Uzunluk • sesli • rakam • boşluk • ters metin • palindrom

Bugünden akılda kalsın

Ezber değil, doğru araç seçimi

[]

tek karakter

[:]

metin parçası

.metot()

dönüştür / sorgula

for

karakterleri dolaş

7. hafta tamamlandı

Metinleri artık yalnızca yazı olarak değil, işlenebilir bir veri yapısı olarak görüyoruz.

Sonraki hafta: ARA SINAV